

final-melhorado-v2

June 8, 2026

```
[3]: '''  
      Bibliotecas usadas  
      '''  
  
      import os  
      import re  
      import subprocess  
      import warnings  
      import numpy as np  
      import pandas as pd  
      from collections import Counter  
      from Bio import Entrez  
      from Bio import SeqIO  
      from Bio.Seq import Seq  
      from sklearn.model_selection import GroupShuffleSplit, StratifiedGroupKFold  
      from sklearn.metrics import (  
          classification_report,  
          roc_auc_score,  
          average_precision_score,  
          matthews_corrcoef,  
          f1_score,  
          precision_score,  
          recall_score,  
          balanced_accuracy_score,  
          confusion_matrix,  
          ConfusionMatrixDisplay,  
          roc_curve,  
          auc  
      )  
  
      from sklearn.ensemble import RandomForestClassifier  
      from sklearn.model_selection import cross_validate  
      from sklearn.svm import SVC  
      from sklearn.cluster import KMeans  
      from sklearn.preprocessing import StandardScaler  
      from xgboost import XGBClassifier  
      import matplotlib.pyplot as plt
```

```
import subprocess
import shap

warnings.filterwarnings("ignore")

print("Bibliotecas importadas")
```

Bibliotecas importadas

```
[4]: '''
    obtenção dos datasets
    '''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineQ"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# =====
# QUERIES
# =====

POSITIVE_QUERY = r'''
("Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR
↳ "Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
AND( TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA OR
↳ PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR
↳ "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR
↳ "Tm-2" OR "RCY1"
OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR
↳ "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
OR "leucine rich repeat receptor kinase" OR "plant disease resistance protein"
↳ OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR "RPS5"
OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" )
NOT ( partial OR fragment OR hypothetical OR predicted OR DAP-seq OR
↳ "transcription factor" OR DNA-binding )
'''

#OR microtom
NEGATIVE_QUERY = r'''
( "Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR
↳ "Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
AND ( "actin" OR "tubulin" OR "ribosomal protein" OR "ATP synthase" OR
↳ "photosystem" OR "elongation factor" OR "glycolysis" OR "metabolic enzyme" )
NOT ( "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR
↳ "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR
↳ "Tm-2"
```

```

OR "RCY1" OR "Rx" OR "R protein" OR "resistance" OR "NLR" OR "LRR" OR "immune"
↳OR "virus" )
'''

# Dados dos transcriptomas
# inclui nos dados de treino
# ARABIDOPSIS_QUERY = '''
#   "Arabidopsis thaliana"[Organism] AND
#     (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR
↳virus OR infected)
#     NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment
↳OR microtom)
#     AND 1500:30000[Sequence Length]
#   '''

SOLANUM_BETACEUM = '''
    Solanum betaceum [Organism]
AND (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR
↳virus OR infected)
    NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment
↳OR microtom)
    AND 1500:30000[Sequence Length]
    '''

SOLANUM_MELONGENA = '''
    Solanum melongena [Organism]
AND (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR
↳virus OR infected)
    NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment
↳OR microtom)
    AND 1500:30000[Sequence Length]
    '''

TUBEROSUM_QUERY = '''
    Solanum tuberosum [Organism]
AND (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR
↳virus OR infected)
    NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment
↳OR microtom)
    AND 1500:30000[Sequence Length]
    '''

PHYSALIS_QUERY = '''
    "Physalis peruviana"[Organism] AND
    (transcriptome OR RNA-Seq OR TSA OR leaf OR fruit OR root OR stress OR
↳virus OR infected)

```

```

        NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment OR_
↳microtom)
        AND 1500:30000[Sequence Length]
    '''

# =====
# DOWNLOAD NCBI
# =====

def baixar_proteinas_ncbi(query, output_fasta, max_records=5000):
    print("\n=====")
    print("DOWNLOAD NCBI")
    print("=====")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n=====")
    print("DOWNLOAD TRANSCRIPTOMA")
    print("=====")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=10000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(

```

```

        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"

baixar_proteinas_ncbi(POSITIVE_QUERY, POSITIVE_FASTA)
baixar_proteinas_ncbi(NEGATIVE_QUERY, NEGATIVE_FASTA)

print("Arquivos salvos na pasta: ", OUTPUT_DIR)
print("===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====")

```

```

=====
DOWNLOAD NCBI
=====
Proteínas encontradas: 5000
Arquivo salvo: pipelineQ/positive.fasta

=====
DOWNLOAD NCBI
=====
Proteínas encontradas: 5000
Arquivo salvo: pipelineQ/negative.fasta
Arquivos salvos na pasta: pipelineQ
===== FIM COLETA DE DADOS TREINO/VALIDAÇÃO =====

```

```

[5]: #Dados para aplicação - baixar transcriptomas - RNA-Seq
# transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
    ↪fasta")
transcriptoma_betaceum_fasta = (f"{OUTPUT_DIR}/betaceum_transcriptoma.fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")

# baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
baixar_transcriptoma(transcriptoma_betaceum_fasta, SOLANUM_BETACEUM)
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)

```

```

baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
print("Arquivos salvos na pasta: ",OUTPUT_DIR)
print("===== FIM COLETA DE DADOS APLICAÇÃO =====")

```

```

=====
DOWNLOAD TRANSCRIPTOMA
=====
Transcritos encontrados: 0
Arquivo salvo: pipelineQ/betaceum_transcriptoma.fasta

=====
DOWNLOAD TRANSCRIPTOMA
=====
Transcritos encontrados: 8589
Arquivo salvo: pipelineQ/melongena_transcriptoma.fasta

=====
DOWNLOAD TRANSCRIPTOMA
=====
Transcritos encontrados: 300
Arquivo salvo: pipelineQ/tuberosum_transcriptoma.fasta

=====
DOWNLOAD TRANSCRIPTOMA
=====
Transcritos encontrados: 10000
Arquivo salvo: pipelineQ/physalis_transcriptoma.fasta
Arquivos salvos na pasta: pipelineQ
===== FIM COLETA DE DADOS APLICAÇÃO =====

```

```

[6]: '''
    Tratamento dos dados
        - Criar Dataframes com os conjuntos de dados de treino e validação
        - Executar CD-Hit para eliminar sequencias muito semelhantes
        - Extração de atributos (features)

    '''

# =====
# AAC
# =====
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas

```

```

# por exemplo proteínas de resistência tem geralmente certa frequência de
↳ aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o
↳ modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# =====
# DIPEPTÍDEOS
# =====
'''
encontrar pares de aminoácidos consecutivos em uma sequência proteica
↳ sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''

DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptídeos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

```

```

]

def extrair_motifs(seq):
    motifs = {
        # "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
        # "GLPL": r"GLPL",
        # "MHD": r"MHD"

        # # novos motifs
        # "KINASE2": r"[LIVM]{3,4}DD[VLIM]",
        # "RNBS_A": r"FDLxKxWVSVSD",
        # "RNBS_B": r"[A-Z]{2}G[PH][A-Z]{2}",
        # "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
        # "RNBS_D": r"CFLYCALFPED",
        # "VVVLDDVW": r"VVVLDDVW"
        "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
        "KINASE2": r"[LIVM]{3,5}DD",
        "RNBS_A": r"FDL",
        "RNBS_B": r"G[RK]G",
        "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
        "GLPL": r"GLPL",
        "RNBS_D": r"CFL",
        "MHD": r"MHD",
        "VVVLDDVW": r"VVVLDDVW"
    }
    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]

# =====
# FEATURES - atributos
# =====
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos -
    ↪ frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera
    ↪ a ordem dos aminoácidos para buscar os mais frequentes em proteínas de
    ↪ resistência

```

```

        features.extend(extrair_motifs(seq))
    return features

# =====
# DATASET
# =====
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "GLPL",
    "RNBS_D",
    "MHD",
    "VVVLDDVW"
]

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

```

```

df = pd.DataFrame(
    data,
    columns=columns + ["target", "id"]
)
print("\nTotal de Features/Atributos:", len(columns)) # quantidade
print("\nFeatures/atributos:")
print(columns)
return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7 #0.5
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id

def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)

```

```

        clusters[cluster_id].append(seq_id)
    return clusters

# executa o CD-Hit nos arquivos baixados
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)

# =====
# Montar os DATASETS
# =====

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"

df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# =====
# CLUSTERS
# =====

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

```

```

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset:", df.shape)

print("Fim tratamento dos dados...")

```

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineQ/positive.fasta -o
         pipelineQ/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Mon Jun 8 13:43:23 2026

```

=====
                        Output
-----

```

```

total seq: 5000
longest and shortest : 2604 and 12
Total letters: 3650305
Sequences have been sorted

```

Approximated minimal memory consumption:

```

Sequence      : 4M
Buffer        : 4 X 11M = 44M
Table         : 2 X 65M = 130M
Miscellaneous  : 0M
Total         : 179M

```

Table limit with the given memory limit:

```

Max number of representatives: 4000000
Max number of word counting entries: 3727547274

```

```

# comparing sequences from          0 to          833
----- new table with          224 representatives
# comparing sequences from          833 to          1527
----- 259 remaining sequences to the next cycle
----- new table with          108 representatives
# comparing sequences from          1268 to          1890
----- 212 remaining sequences to the next cycle

```

```

----- new table with      100 representatives
# comparing sequences from      1678 to      2231
----- 197 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      2034 to      2528
----- 153 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      2375 to      2812
----- 134 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      2678 to      3065
----- 6 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from      3059 to      3382
...----- new table with      69 representatives
# comparing sequences from      3382 to      3651
...----- new table with      90 representatives
# comparing sequences from      3651 to      3875
...----- new table with      74 representatives
# comparing sequences from      3875 to      4062
...----- new table with      56 representatives
# comparing sequences from      4062 to      5000
...----- new table with      284 representatives

```

5000 finished 1405 clusters

Approximated maximum memory consumption: 182M
writing new database
writing clustering information
program completed !

Total CPU time 1.17
CD-HIT concluído.

Executando CD-HIT...

```

=====
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineQ/negative.fasta -o
         pipelineQ/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

```

Started: Mon Jun 8 13:43:24 2026

```

=====
                        Output
-----

```

```

total seq: 5000
longest and shortest : 5098 and 25
Total letters: 1563139
Sequences have been sorted

```

Approximated minimal memory consumption:

Sequence : 2M
Buffer : 4 X 11M = 46M
Table : 2 X 65M = 130M
Miscellaneous : 0M
Total : 179M

Table limit with the given memory limit:

Max number of representatives: 4000000
Max number of word counting entries: 3727542149

```
# comparing sequences from      0 to      833
----- new table with      392 representatives
# comparing sequences from      833 to     1527
-----      323 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from     1204 to     1836
-----      296 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from     1540 to     2116
-----      296 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from     1820 to     2350
-----      97 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from     2253 to     2710
-----      91 remaining sequences to the next cycle
----- new table with      100 representatives
# comparing sequences from     2619 to     3015
...----- new table with      100 representatives
# comparing sequences from     3015 to     3345
...----- new table with      95 representatives
# comparing sequences from     3345 to     3620
...----- new table with      60 representatives
# comparing sequences from     3620 to     3850
...----- new table with      46 representatives
# comparing sequences from     3850 to     4041
...----- new table with      30 representatives
# comparing sequences from     4041 to     5000
...----- new table with     176 representatives
```

5000 finished 1399 clusters

Approximated maximum memory consumption: 182M

writing new database

writing clustering information

program completed !

Total CPU time 0.45

CD-HIT concluído.

Total de Features/Atributos: 430

Features/atributos:

['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',

```

'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',
'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'RNBS_A', 'RNBS_B', 'RNBS_C', 'GLPL', 'RNBS_D',
'MHD', 'VVVLDDVW']

```

Clusters ausentes:

705

Dataset: (2085, 433)

Fim tratamento dos dados...

```

[7]: '''
      k-Fold-Cross-Validation

      '''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=5, #10
        shuffle=True,

```

```

        random_state=42
    )

    scores = cross_validate(
        model,
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=[
            'accuracy',
            'precision',
            'recall',
            'f1',
            'roc_auc'
        ],
        n_jobs=-1
    )
    return scores

```

```

[15]: '''
Criação dos modelos/ Treinamento
'''

# para melhor separar os conjuntos de treino e teste considerando os
# agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

```

```

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos
    explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,
    valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
        alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência
    aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20, # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",
    dpi=300)
    plt.close()

```

```

# 5. Gráfico 2: Bar Plot (Importância Global)
# Mostra a magnitude média do impacto de cada feature de forma absoluta
plt.figure(figsize=(12, 8))
shap.summary_plot(
    shap_values_pos,
    X_test,
    feature_names=feature_names,
    plot_type="bar",
    max_display=20,
    show=False
)
plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",
    ↪dpi=300)
plt.close()

print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")

# =====
# TREINAMENTO - treina os modelos
# =====

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n=====")
    print(nome)
    print("=====")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[: ,1]
    preds = (probs >= 0.5).astype(int)
    print(classification_report(y_test, preds))
    roc = roc_auc_score(y_test, probs)
    pr = average_precision_score(y_test, probs)
    mcc = matthews_corrcoef(y_test, preds)
    f1 = f1_score(y_test, preds)
    precision = precision_score(y_test, preds)
    recall = recall_score(y_test, preds)
    bal_acc = balanced_accuracy_score(y_test, preds)

    print("ROC-AUC          :", roc)
    print("PR-AUC           :", pr)
    print("MCC              :", mcc)
    print("F1               :", f1)
    print("Precision        :", precision)
    print("Recall           :", recall)
    print("Balanced Accuracy:", bal_acc)

```

```

# =====
# MATRIZ DE CONFUSÃO
# =====
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(5,5))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativo", "Positivo"]
)
disp.plot(cmap="Blues")
# plt.title(f"Matriz de Confusão - {nome}")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
    bbox_inches="tight"
)
plt.close()
# =====
# CURVA ROC
# =====
fpr, tpr, _ = roc_curve(y_test, probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))
plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.4f}"
)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# plt.title(f"Curva ROC - {nome}")
plt.legend(loc="lower right")
plt.savefig(
    f"{OUTPUT_DIR}/{nome}_roc.png",
    bbox_inches="tight"
)
plt.close()

return {
    "Modelo": nome,
    "ROC_AUC": roc,
    "F1": f1,
    "Precision": precision,
    "Recall": recall,
    "Balanced_Accuracy": bal_acc
}, model

```

```

# =====
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# =====

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# MODELOS
# =====

rf = RandomForestClassifier(
    n_estimators=1000,
    max_depth = 12,
    min_samples_split=4,
    max_features='sqrt',
    random_state=42,
    class_weight="balanced",
    n_jobs = -1
)
xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=5,
    learning_rate=0.01,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.2
)
svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,

```

```

        C=2,
        gamma='scale'
    )
    scores = []
    groups_train = train_df["cluster"]

    scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
    scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
    scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

    print("Cross-Validation:RF")
    for k,v in scores_rf.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

    print("Cross-Validation XGB")
    for k,v in scores_xgb.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

    print("Cross-Validation SVM")
    for k,v in scores_svm.items():
        if "test" in k:
            print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

resultados_cv = pd.DataFrame({
    'Modelo': ['RF', 'XGB', 'SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
})

```

```

    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ]
})
print(resultados_cv)

dados = []

for score in scores_rf['test_roc_auc']:
    dados.append(['RF',score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB',score])
for score in scores_svm['test_roc_auc']:
    dados.append(['SVM',score])

grafico = pd.DataFrame(dados, columns=['Modelo','ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()

#####
# Treinamento

results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns,OUTPUT_DIR )

```

```

r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns, OUTPUT_DIR )

r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r3)

print("\nFim criação/treino dos modelos\n")

```

	protein_length	AAC_A	AAC_C	AAC_D	AAC_E	AAC_F	\
0	1053	0.034188	0.018993	0.043685	0.038936	0.058879	
1	231	0.090909	0.012987	0.069264	0.077922	0.051948	
2	888	0.055180	0.018018	0.061937	0.072072	0.045045	
3	504	0.103175	0.019841	0.045635	0.071429	0.029762	
4	704	0.049716	0.007102	0.080966	0.117898	0.041193	
5	535	0.095327	0.009346	0.020561	0.026168	0.082243	
6	450	0.040000	0.028889	0.035556	0.082222	0.024444	
7	117	0.025641	0.017094	0.051282	0.068376	0.042735	
8	89	0.123596	0.078652	0.011236	0.112360	0.033708	
96	751	0.061252	0.014647	0.057257	0.055925	0.038615	
97	166	0.072289	0.024096	0.054217	0.072289	0.054217	
98	160	0.043750	0.062500	0.087500	0.062500	0.062500	
99	116	0.034483	0.086207	0.120690	0.068966	0.086207	
100	1167	0.032562	0.011140	0.084833	0.117395	0.024850	
101	812	0.036946	0.027094	0.060345	0.081281	0.040640	
102	782	0.061381	0.015345	0.066496	0.070332	0.034527	
103	896	0.053571	0.016741	0.049107	0.050223	0.046875	
104	1038	0.069364	0.011561	0.065511	0.075145	0.049133	
105	334	0.080838	0.002994	0.047904	0.077844	0.029940	
106	214	0.116822	0.000000	0.074766	0.112150	0.028037	
107	665	0.046617	0.022556	0.049624	0.052632	0.049624	
108	448	0.055804	0.033482	0.062500	0.037946	0.024554	
109	182	0.060440	0.065934	0.076923	0.087912	0.049451	

110	909	0.044004	0.028603	0.037404	0.097910	0.036304
111	540	0.044444	0.022222	0.051852	0.046296	0.044444
112	790	0.060759	0.026582	0.054430	0.035443	0.056962
113	830	0.059036	0.027711	0.053012	0.036145	0.059036
114	690	0.057971	0.023188	0.056522	0.043478	0.052174
115	616	0.068182	0.022727	0.055195	0.037338	0.056818
116	660	0.059091	0.025758	0.050000	0.054545	0.057576

	AAC_G	AAC_H	AAC_I	AAC_K	...	RNBS_A	RNBS_B	RNBS_C	\
0	0.059829	0.017094	0.056980	0.036087	...	0	0	0	
1	0.077922	0.038961	0.043290	0.082251	...	0	0	0	
2	0.047297	0.023649	0.061937	0.085586	...	0	0	1	
3	0.081349	0.005952	0.075397	0.073413	...	0	0	0	
4	0.048295	0.009943	0.065341	0.117898	...	1	1	0	
5	0.106542	0.014953	0.065421	0.039252	...	0	0	1	
6	0.048889	0.006667	0.075556	0.091111	...	0	0	0	
7	0.076923	0.119658	0.042735	0.059829	...	0	0	0	
8	0.078652	0.056180	0.022472	0.089888	...	0	0	0	
96	0.078562	0.017310	0.053262	0.049268	...	0	0	1	
97	0.036145	0.024096	0.054217	0.090361	...	0	0	0	
98	0.043750	0.037500	0.025000	0.062500	...	0	0	0	
99	0.017241	0.034483	0.025862	0.060345	...	0	0	0	
100	0.065124	0.014567	0.046272	0.140531	...	0	1	1	
101	0.049261	0.029557	0.064039	0.062808	...	1	0	1	
102	0.083120	0.019182	0.056266	0.078005	...	0	1	1	
103	0.062500	0.021205	0.056920	0.070312	...	0	1	0	
104	0.043353	0.020231	0.058767	0.053950	...	0	0	0	
105	0.101796	0.026946	0.080838	0.050898	...	0	1	0	
106	0.093458	0.000000	0.037383	0.116822	...	0	0	0	
107	0.063158	0.018045	0.064662	0.058647	...	0	0	0	
108	0.049107	0.024554	0.029018	0.044643	...	0	0	0	
109	0.027473	0.032967	0.032967	0.104396	...	0	0	0	
110	0.050605	0.024202	0.057206	0.059406	...	0	0	0	
111	0.072222	0.020370	0.072222	0.053704	...	0	0	0	
112	0.056962	0.016456	0.053165	0.034177	...	0	1	0	
113	0.056627	0.015663	0.056627	0.037349	...	0	1	0	
114	0.069565	0.023188	0.063768	0.047826	...	0	0	0	
115	0.063312	0.006494	0.060065	0.038961	...	1	1	0	
116	0.063636	0.016667	0.077273	0.037879	...	0	1	1	

	GLPL	RNBS_D	MHD	VVVLDDVW	target	id	cluster
0	1	0	0	0	1	NP_001234474.3	POS_257
1	0	0	0	0	1	NP_001234459.3	POS_1230
2	1	1	0	0	1	NP_001234202.1	POS_465
3	0	0	0	0	1	NP_001333606.1	POS_919
4	0	0	0	0	1	NP_001308492.1	POS_700
5	0	0	0	0	1	NP_001307892.1	POS_896
6	0	0	0	0	1	NP_001234240.1	POS_973

7	0	0	0	0	1	pdb 9KBZ B	POS_1375
8	0	0	0	0	1	pdb 9K9P A	POS_1387
96	0	0	0	0	1	CA07058141.1	POS_647
97	0	0	0	0	1	CA07056953.1	POS_1343
98	1	0	0	0	1	CA07056440.1	POS_1347
99	0	0	0	0	1	CA07056439.1	POS_1376
100	0	0	0	0	1	CA07055314.1	POS_184
101	0	0	0	0	1	CA07055273.1	POS_582
102	0	0	0	0	1	CA07054023.1	POS_614
103	0	0	0	0	1	CA07053068.1	POS_435
104	0	0	0	0	1	CA07052349.1	POS_272
105	0	0	0	0	1	CA07051023.1	POS_1127
106	0	0	0	0	1	CA07050796.1	POS_1273
107	0	0	0	0	1	CA07050404.1	POS_770
108	0	0	0	0	1	CA07049100.1	POS_976
109	0	0	0	0	1	CA07048966.1	POS_1327
110	1	1	0	0	1	CA07048770.1	POS_418
111	0	0	0	0	1	CA07048398.1	POS_892
112	0	0	0	0	1	CA07048397.1	POS_601
113	0	0	0	0	1	CA07048396.1	POS_561
114	0	0	0	0	1	CA07048394.1	POS_721
115	0	0	0	0	1	CA07048391.1	POS_844
116	0	0	0	0	1	CA07048379.1	POS_784

[30 rows x 433 columns]

Cross-Validation:RF

test_accuracy: 0.8387 ± 0.0102

test_precision: 0.8797 ± 0.0104

test_recall: 0.8098 ± 0.0101

test_f1: 0.8433 ± 0.0100

test_roc_auc: 0.9292 ± 0.0070

Cross-Validation XGB

test_accuracy: 0.8717 ± 0.0062

test_precision: 0.8901 ± 0.0090

test_recall: 0.8680 ± 0.0179

test_f1: 0.8787 ± 0.0070

test_roc_auc: 0.9450 ± 0.0060

Cross-Validation SVM

test_accuracy: 0.8717 ± 0.0130

test_precision: 0.9168 ± 0.0115

test_recall: 0.8367 ± 0.0248

test_f1: 0.8747 ± 0.0141

test_roc_auc: 0.9468 ± 0.0063

	Modelo	Accuracy	Recall	Precision	F1	ROC_AUC
0	RF	0.838720	0.809836	0.879708	0.843320	0.929172
1	XGB	0.871698	0.868006	0.890119	0.878746	0.945025
2	SVM	0.871701	0.836702	0.916825	0.874692	0.946769

```
=====
Random Forest
=====
```

	precision	recall	f1-score	support
0	0.83	0.88	0.86	196
1	0.89	0.85	0.87	221
accuracy			0.86	417
macro avg	0.86	0.86	0.86	417
weighted avg	0.86	0.86	0.86	417

```
ROC-AUC      : 0.9310531905069721
PR-AUC       : 0.948043850331561
MCC          : 0.7224550443140061
F1           : 0.8657407407407407
Precision    : 0.8862559241706162
Recall       : 0.8461538461538461
Balanced Accuracy: 0.8618524332810047
```

```
[SHAP] Inicializando análise de interpretabilidade para: Random Forest...
[SHAP] Gráficos salvos com sucesso em 'pipelineQ/'!
```

```
=====
XGBoost
=====
```

	precision	recall	f1-score	support
0	0.87	0.89	0.88	196
1	0.90	0.88	0.89	221
accuracy			0.88	417
macro avg	0.88	0.89	0.88	417
weighted avg	0.89	0.88	0.88	417

```
ROC-AUC      : 0.9453204358666544
PR-AUC       : 0.96225476076757
MCC          : 0.7696730414654839
F1           : 0.8899082568807339
Precision    : 0.9023255813953488
Recall       : 0.8778280542986425
Balanced Accuracy: 0.8853425985778927
```

```
[SHAP] Inicializando análise de interpretabilidade para: XGBoost...
[SHAP] Gráficos salvos com sucesso em 'pipelineQ/'!
```

```
=====
SVM
=====
```

```
=====
              precision    recall  f1-score   support

     0           0.88        0.93        0.91        196
     1           0.94        0.89        0.91        221

 accuracy                   0.91        417
 macro avg           0.91        0.91        0.91        417
 weighted avg        0.91        0.91        0.91        417

ROC-AUC      : 0.9479984301412873
PR-AUC       : 0.9666166087652677
MCC          : 0.823613400023702
F1           : 0.91415313225058
Precision    : 0.9380952380952381
Recall       : 0.8914027149321267
Balanced Accuracy: 0.9125380921599409
```

Fim criação/treino dos modelos

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

```
[16]: '''
Avaliação e Validação dos modelos
'''

# verificar importância dos atributos/feature para o Random Forest
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
    feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][:-1], top["importance"][:-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()
```

```

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
    ↳feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

```

```

plt.savefig(
    f"{OUTPUT_DIR}/comparacao_{metric}.png",
    bbox_inches="tight"
)
plt.close()

modelos = {
    "Random Forest": rf_model,
    "XGBoost": xgb_model,
    "SVM": svm_model
}

plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[: ,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1], [0,1], '--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")

```

Top 20 features: Random Forest

	feature	importance
0	protein_length	0.032332
1	AAC_A	0.031940
10	AAC_L	0.020408
207	DIPEP_LH	0.018931
250	DIPEP_NL	0.017609
422	KINASE2	0.013113
65	DIPEP_DF	0.012780
106	DIPEP_FG	0.012756
50	DIPEP_CL	0.012366
421	P_LOOP	0.011541
100	DIPEP_EY	0.011392
410	DIPEP_YL	0.011127
272	DIPEP_PN	0.010040
248	DIPEP_NI	0.009991

21	DIPEP_AA	0.009592
85	DIPEP_EF	0.009402
210	DIPEP_LL	0.009171
167	DIPEP_IH	0.008588
45	DIPEP_CF	0.008304
380	DIPEP_VY	0.008141

Top 20 features: XGBoost

	feature	importance
0	protein_length	0.060213
422	KINASE2	0.016675
207	DIPEP_LH	0.011878
1	AAC_A	0.011428
250	DIPEP_NL	0.008655
10	AAC_L	0.007536
155	DIPEP_HR	0.007191
421	P_LOOP	0.007105
402	DIPEP_YC	0.006613
42	DIPEP_CC	0.006125
426	GLPL	0.006047
100	DIPEP_EY	0.005581
106	DIPEP_FG	0.005527
154	DIPEP_HQ	0.005347
92	DIPEP_EN	0.005281
390	DIPEP_WL	0.004854
411	DIPEP_YM	0.004749
50	DIPEP_CL	0.004714
210	DIPEP_LL	0.004673
386	DIPEP_WG	0.004556

COMPARAÇÃO MODELOS

	Modelo	ROC_AUC	F1	Precision	Recall	Balanced_Accuracy
0	Random Forest	0.931053	0.865741	0.886256	0.846154	0.861852
1	XGBoost	0.945320	0.889908	0.902326	0.877828	0.885343
2	SVM	0.947998	0.914153	0.938095	0.891403	0.912538

Fim avaliação / validação modelos

```
[11]: '''
Funções para rodar a aplicação
'''

MOTIFS = {
    # "P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
    # "GLPL": r"GLPL",
    # "MHD": r"MHD"

    # # novos motivos
    # "KINASE2": r"[LIVM]{3,4}DD[VLIM]",
```

```

# "RNBS_A": r"FDLxKxWVSVD",
# "RNBS_B": r"[A-Z]{2}G[PH][A-Z]{2}",
# "RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
# "RNBS_D": r"CFLYCALFPED",
# "VVVLDDVW": r"VVVLDDVW"
"P_LOOP": r"[A-Z]G[A-Z]{4}GK[ST]",
"KINASE2": r"[LIVM]{3,5}DD",
"RNBS_A": r"FDL",
"RNBS_B": r"G[RK]G",
"RNBS_C": r"[A-Z]{2}P[A-Z]{2}W",
"GLPL": r"GLPL",
"RNBS_D": r"CFL",
"MHD": r"MHD",
"VVVLDDVW": r"VVVLDDVW"
}

def validar_motifs(df):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in MOTIFS.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",

```

```

        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                        stdout=log_file,
                        stderr=subprocess.STDOUT,
                        text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file

def predizer_transcritos(fasta_nt,model,nmodel, output):
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    top = candidatos_df.head(20)
    print("\nTOP CANDIDATOS: "+fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])

    validar_motifs(top)

    candidatos_df.to_csv(

```

```

f"{OUTPUT_DIR}/{output}",
index=False
)

```

```

[12]: '''
Aplicação - Modelo XGBoost
'''

# print("\nPredição para ARABIDOPSIS\n")
# predizer_transcritos(transcriptoma_arabidopsis_fasta,
# ↪xgb_model, 'XGBoost', output="predicoes_xgboost.csv")

# print("\nPredição para BETACEUM\n")
# predizer_transcritos(transcriptoma_betaceum_fasta,
# ↪xgb_model, 'XGBoost', output="predicoes_xgboost.csv")

print("\nPredição para MELONGENA\n")
predizer_transcritos(transcriptoma_melongena_fasta,
↪xgb_model, 'XGBoost', output="predicoes_xgboost.csv")

print("\nPredição para TUBEROSUM\n")
predizer_transcritos(transcriptoma_tuberosum_fasta,
↪xgb_model, 'XGBoost', output="predicoes_xgboost.csv")

print("\nPredição para PHYSALIS\n")
predizer_transcritos(transcriptoma_physalis_fasta,
↪xgb_model, 'XGBoost', output="predicoes_xgboost.csv")

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")

```

Predição para MELONGENA

```

TOP CANDIDATOS: pipelineQ/melongena_transcriptoma.fasta, MODELO: XGBoost

```

	id	descricao \
3774	GBEF01024584.1.p1	GBEF01024584.1.p1 GENE.GBEF01024584.1~~GBEF010...
40	GBEF01000145.1.p1	GBEF01000145.1.p1 GENE.GBEF01000145.1~~GBEF010...
994	GBEF01014969.1.p1	GBEF01014969.1.p1 GENE.GBEF01014969.1~~GBEF010...
8171	GBEF01041955.1.p1	GBEF01041955.1.p1 GENE.GBEF01041955.1~~GBEF010...
5210	GBEF01027867.1.p1	GBEF01027867.1.p1 GENE.GBEF01027867.1~~GBEF010...
286	GBEF01001123.1.p1	GBEF01001123.1.p1 GENE.GBEF01001123.1~~GBEF010...
589	GBEF01002241.1.p1	GBEF01002241.1.p1 GENE.GBEF01002241.1~~GBEF010...
36	GBEF01000129.1.p1	GBEF01000129.1.p1 GENE.GBEF01000129.1~~GBEF010...
590	GBEF01002242.1.p1	GBEF01002242.1.p1 GENE.GBEF01002242.1~~GBEF010...
6324	GBEF01032168.1.p1	GBEF01032168.1.p1 GENE.GBEF01032168.1~~GBEF010...

9150	GBEF01044247.1.p1	GBEF01044247.1.p1	GENE.GBEF01044247.1~~GBEF010...
4127	GBEF01025346.1.p1	GBEF01025346.1.p1	GENE.GBEF01025346.1~~GBEF010...
967	GBEF01014919.1.p1	GBEF01014919.1.p1	GENE.GBEF01014919.1~~GBEF010...
5625	GBEF01028897.1.p1	GBEF01028897.1.p1	GENE.GBEF01028897.1~~GBEF010...
518	GBEF01002012.1.p1	GBEF01002012.1.p1	GENE.GBEF01002012.1~~GBEF010...
519	GBEF01002013.1.p1	GBEF01002013.1.p1	GENE.GBEF01002013.1~~GBEF010...
3816	GBEF01024662.1.p1	GBEF01024662.1.p1	GENE.GBEF01024662.1~~GBEF010...
1568	GBEF01016235.1.p1	GBEF01016235.1.p1	GENE.GBEF01016235.1~~GBEF010...
9151	GBEF01044248.1.p1	GBEF01044248.1.p1	GENE.GBEF01044248.1~~GBEF010...
8760	GBEF01043405.1.p1	GBEF01043405.1.p1	GENE.GBEF01043405.1~~GBEF010...

	protein_length	probabilidade_resistencia
3774	1214	0.997964
40	808	0.997299
994	1020	0.997184
8171	959	0.996850
5210	965	0.996707
286	749	0.996691
589	815	0.996508
36	718	0.996267
590	802	0.996233
6324	679	0.996146
9150	713	0.995833
4127	1022	0.994814
967	978	0.994572
5625	905	0.994398
518	727	0.994101
519	723	0.994045
3816	462	0.994045
1568	982	0.993944
9151	706	0.993893
8760	921	0.993778

VALIDAÇÃO BIOLÓGICA

P_LOOP : 0
 KINASE2 : 5
 RNBS_A : 2
 RNBS_B : 10
 RNBS_C : 5
 GLPL : 2
 RNBS_D : 2
 MHD : 0
 VVVLDDVW : 0

Predição para TUBEROSUM

TOP CANDIDATOS: pipelineQ/tuberosum_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...
43	BD391474.1.p1	BD391474.1.p1 GENE.BD391474.1~~BD391474.1.p1 ...
211	MG763883.1.p1	MG763883.1.p1 GENE.MG763883.1~~MG763883.1.p1 ...
46	EF471737.1.p1	EF471737.1.p1 GENE.EF471737.1~~EF471737.1.p1 ...
20	AJ300266.1.p1	AJ300266.1.p1 GENE.AJ300266.1~~AJ300266.1.p1 ...
269	PP061819.1.p1	PP061819.1.p1 GENE.PP061819.1~~PP061819.1.p1 ...
24	AJ306627.1.p1	AJ306627.1.p1 GENE.AJ306627.1~~AJ306627.1.p1 ...
23	AJ306626.1.p1	AJ306626.1.p1 GENE.AJ306626.1~~AJ306626.1.p1 ...
258	PP061811.1.p1	PP061811.1.p1 GENE.PP061811.1~~PP061811.1.p1 ...
163	KC914392.1.p1	KC914392.1.p1 GENE.KC914392.1~~KC914392.1.p1 ...
152	JE864385.1.p1	JE864385.1.p1 GENE.JE864385.1~~JE864385.1.p1 ...
78	HI543298.1.p1	HI543298.1.p1 GENE.HI543298.1~~HI543298.1.p1 ...
162	KC914391.1.p1	KC914391.1.p1 GENE.KC914391.1~~KC914391.1.p1 ...
271	PP061822.1.p1	PP061822.1.p1 GENE.PP061822.1~~PP061822.1.p1 ...
249	PP061795.1.p1	PP061795.1.p1 GENE.PP061795.1~~PP061795.1.p1 ...
263	PP061818.1.p1	PP061818.1.p1 GENE.PP061818.1~~PP061818.1.p1 ...
256	PP061808.1.p1	PP061808.1.p1 GENE.PP061808.1~~PP061808.1.p1 ...
137	JC300327.1.p1	JC300327.1.p1 GENE.JC300327.1~~JC300327.1.p1 ...
128	JC300104.1.p1	JC300104.1.p1 GENE.JC300104.1~~JC300104.1.p1 ...
164	KC979135.1.p1	KC979135.1.p1 GENE.KC979135.1~~KC979135.1.p1 ...

	protein_length	probabilidade_resistencia
45	849	0.997681
43	871	0.996167
211	895	0.995295
46	1207	0.995271
20	486	0.981988
269	337	0.980313
24	677	0.975914
23	677	0.973883
258	471	0.973657
163	616	0.970850
152	427	0.970203
78	427	0.970203
162	616	0.968246
271	478	0.966122
249	478	0.964273
263	478	0.963883
256	478	0.963883
137	483	0.960934
128	483	0.960934
164	1018	0.955821

VALIDAÇÃO BIOLÓGICA

P_LOOP : 3

KINASE2 : 2
 RNBS_A : 5
 RNBS_B : 4
 RNBS_C : 6
 GLPL : 2
 RNBS_D : 3
 MHD : 1
 VVVLDDVW : 0

Predição para PHYSALIS

TOP CANDIDATOS: pipelineQ/physalis_transcriptoma.fasta, MODELO: XGBoost

	id	descricao \
8855	J0132548.1.p1	J0132548.1.p1 GENE.J0132548.1~~J0132548.1.p1 ...
941	J0125565.1.p1	J0125565.1.p1 GENE.J0125565.1~~J0125565.1.p1 ...
8857	J0132549.1.p1	J0132549.1.p1 GENE.J0132549.1~~J0132549.1.p1 ...
8854	J0132547.1.p1	J0132547.1.p1 GENE.J0132547.1~~J0132547.1.p1 ...
12659	J0138542.1.p1	J0138542.1.p1 GENE.J0138542.1~~J0138542.1.p1 ...
12682	J0138557.1.p1	J0138557.1.p1 GENE.J0138557.1~~J0138557.1.p1 ...
8851	J0132546.1.p1	J0132546.1.p1 GENE.J0132546.1~~J0132546.1.p1 ...
10869	J0134211.1.p1	J0134211.1.p1 GENE.J0134211.1~~J0134211.1.p1 ...
10868	J0134210.1.p1	J0134210.1.p1 GENE.J0134210.1~~J0134210.1.p1 ...
9448	J0133080.1.p1	J0133080.1.p1 GENE.J0133080.1~~J0133080.1.p1 ...
13394	J0139076.1.p1	J0139076.1.p1 GENE.J0139076.1~~J0139076.1.p1 ...
12825	J0138656.1.p1	J0138656.1.p1 GENE.J0138656.1~~J0138656.1.p1 ...
12711	J0138575.1.p1	J0138575.1.p1 GENE.J0138575.1~~J0138575.1.p1 ...
12868	J0138683.1.p1	J0138683.1.p1 GENE.J0138683.1~~J0138683.1.p1 ...
10739	J0134126.1.p1	J0134126.1.p1 GENE.J0134126.1~~J0134126.1.p1 ...
2142	J0126348.1.p1	J0126348.1.p1 GENE.J0126348.1~~J0126348.1.p1 ...
2144	J0126350.1.p1	J0126350.1.p1 GENE.J0126350.1~~J0126350.1.p1 ...
10742	J0134127.1.p1	J0134127.1.p1 GENE.J0134127.1~~J0134127.1.p1 ...
9437	J0133072.1.p1	J0133072.1.p1 GENE.J0133072.1~~J0133072.1.p1 ...
9438	J0133073.1.p1	J0133073.1.p1 GENE.J0133073.1~~J0133073.1.p1 ...

	protein_length	probabilidade_resistencia
8855	639	0.998636
941	646	0.998547
8857	622	0.998231
8854	759	0.998151
12659	1068	0.998150
12682	888	0.998149
8851	776	0.998091
10869	865	0.997802
10868	865	0.997776
9448	642	0.997609
13394	674	0.997481
12825	1150	0.997113

12711	1214	0.996789
12868	1148	0.996361
10739	628	0.996320
2142	873	0.996303
2144	873	0.996303
10742	628	0.996246
9437	938	0.996194
9438	938	0.996194

VALIDAÇÃO BIOLÓGICA

P_LOOP : 12
 KINASE2 : 13
 RNBS_A : 0
 RNBS_B : 4
 RNBS_C : 6
 GLPL : 13
 RNBS_D : 12
 MHD : 2
 VVVLDDVW : 0

Arquivo salvo:
 pipelineQ/predicoes_xgboost.csv

```
[13]: '''
Aplicação - Modelo Random Forest
'''

# print("\nPredição para ARABIDOPSIS\n")
# predizer_transcritos(transcriptoma_arabidopsis_fasta,
# ↪rf_model, "RF", output="predicoes_rf.csv")

# print("\nPredição para BETACEUM\n")
# predizer_transcritos(transcriptoma_betaceum_fasta,
# ↪rf_model, 'RF', output="predicoes_rf.csv")

print("\nPredição para MELONGENA\n")
predizer_transcritos(transcriptoma_melongena_fasta,
↪rf_model, 'RF', output="predicoes_rf.csv")

print("\nPredição para TUBEROSUM\n")
predizer_transcritos(transcriptoma_tuberosum_fasta,
↪rf_model, "RF", output="predicoes_rf.csv")

print("\nPredição para PHYSALIS\n")
predizer_transcritos(transcriptoma_physalis_fasta,
↪rf_model, "RF", output="predicoes_rf.csv")
```

```
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

Predição para MELONGENA

TOP CANDIDATOS: pipelineQ/melongena_transcriptoma.fasta, MODELO: RF

	id	descricao \
267	GBEF01001042.1.p1	GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010...
1568	GBEF01016235.1.p1	GBEF01016235.1.p1 GENE.GBEF01016235.1~~GBEF010...
589	GBEF01002241.1.p1	GBEF01002241.1.p1 GENE.GBEF01002241.1~~GBEF010...
590	GBEF01002242.1.p1	GBEF01002242.1.p1 GENE.GBEF01002242.1~~GBEF010...
994	GBEF01014969.1.p1	GBEF01014969.1.p1 GENE.GBEF01014969.1~~GBEF010...
4989	GBEF01027318.1.p1	GBEF01027318.1.p1 GENE.GBEF01027318.1~~GBEF010...
5625	GBEF01028897.1.p1	GBEF01028897.1.p1 GENE.GBEF01028897.1~~GBEF010...
8171	GBEF01041955.1.p1	GBEF01041955.1.p1 GENE.GBEF01041955.1~~GBEF010...
36	GBEF01000129.1.p1	GBEF01000129.1.p1 GENE.GBEF01000129.1~~GBEF010...
9150	GBEF01044247.1.p1	GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010...
3774	GBEF01024584.1.p1	GBEF01024584.1.p1 GENE.GBEF01024584.1~~GBEF010...
40	GBEF01000145.1.p1	GBEF01000145.1.p1 GENE.GBEF01000145.1~~GBEF010...
6390	GBEF01032474.1.p1	GBEF01032474.1.p1 GENE.GBEF01032474.1~~GBEF010...
5647	GBEF01028998.1.p1	GBEF01028998.1.p1 GENE.GBEF01028998.1~~GBEF010...
4481	GBEF01026076.1.p1	GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010...
4482	GBEF01026077.1.p1	GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010...
3214	GBEF01023314.1.p1	GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010...
5210	GBEF01027867.1.p1	GBEF01027867.1.p1 GENE.GBEF01027867.1~~GBEF010...
4127	GBEF01025346.1.p1	GBEF01025346.1.p1 GENE.GBEF01025346.1~~GBEF010...
8760	GBEF01043405.1.p1	GBEF01043405.1.p1 GENE.GBEF01043405.1~~GBEF010...

	protein_length	probabilidade_resistencia
267	867	0.948680
1568	982	0.935175
589	815	0.929916
590	802	0.924139
994	1020	0.922729
4989	935	0.922255
5625	905	0.921998
8171	959	0.921289
36	718	0.919026
9150	713	0.918981
3774	1214	0.918587
40	808	0.917915
6390	905	0.914007
5647	1005	0.906293
4481	892	0.903042
4482	866	0.901008
3214	872	0.900683

5210	965	0.897769
4127	1022	0.895764
8760	921	0.894213

VALIDAÇÃO BIOLÓGICA

P_LOOP : 4
 KINASE2 : 8
 RNBS_A : 2
 RNBS_B : 14
 RNBS_C : 7
 GLPL : 4
 RNBS_D : 5
 MHD : 0
 VVVLDDVW : 0

Predição para TUBEROSUM

TOP CANDIDATOS: pipelineQ/tuberosum_transcriptoma.fasta, MODELO: RF

	id	descricao \
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...
43	BD391474.1.p1	BD391474.1.p1 GENE.BD391474.1~~BD391474.1.p1 ...
46	EF471737.1.p1	EF471737.1.p1 GENE.EF471737.1~~EF471737.1.p1 ...
211	MG763883.1.p1	MG763883.1.p1 GENE.MG763883.1~~MG763883.1.p1 ...
24	AJ306627.1.p1	AJ306627.1.p1 GENE.AJ306627.1~~AJ306627.1.p1 ...
23	AJ306626.1.p1	AJ306626.1.p1 GENE.AJ306626.1~~AJ306626.1.p1 ...
74	HI543289.1.p1	HI543289.1.p1 GENE.HI543289.1~~HI543289.1.p1 ...
148	JE864376.1.p1	JE864376.1.p1 GENE.JE864376.1~~JE864376.1.p1 ...
163	KC914392.1.p1	KC914392.1.p1 GENE.KC914392.1~~KC914392.1.p1 ...
162	KC914391.1.p1	KC914391.1.p1 GENE.KC914391.1~~KC914391.1.p1 ...
164	KC979135.1.p1	KC979135.1.p1 GENE.KC979135.1~~KC979135.1.p1 ...
48	EU049848.1.p1	EU049848.1.p1 GENE.EU049848.1~~EU049848.1.p1 ...
20	AJ300266.1.p1	AJ300266.1.p1 GENE.AJ300266.1~~AJ300266.1.p1 ...
249	PP061795.1.p1	PP061795.1.p1 GENE.PP061795.1~~PP061795.1.p1 ...
256	PP061808.1.p1	PP061808.1.p1 GENE.PP061808.1~~PP061808.1.p1 ...
263	PP061818.1.p1	PP061818.1.p1 GENE.PP061818.1~~PP061818.1.p1 ...
173	KT036486.1.p1	KT036486.1.p1 GENE.KT036486.1~~KT036486.1.p1 ...
258	PP061811.1.p1	PP061811.1.p1 GENE.PP061811.1~~PP061811.1.p1 ...
271	PP061822.1.p1	PP061822.1.p1 GENE.PP061822.1~~PP061822.1.p1 ...
239	OM569620.1.p1	OM569620.1.p1 GENE.OM569620.1~~OM569620.1.p1 ...

	protein_length	probabilidade_resistencia
45	849	0.947821
43	871	0.933342
46	1207	0.924134
211	895	0.913357
24	677	0.876791

23	677	0.845820
74	886	0.814267
148	886	0.814267
163	616	0.808169
162	616	0.791520
164	1018	0.788525
48	797	0.782185
20	486	0.767266
249	478	0.749445
256	478	0.747667
263	478	0.747667
173	618	0.734148
258	471	0.719835
271	478	0.715650
239	580	0.699885

VALIDAÇÃO BIOLÓGICA

P_LOOP : 2
 KINASE2 : 2
 RNBS_A : 3
 RNBS_B : 2
 RNBS_C : 4
 GLPL : 1
 RNBS_D : 2
 MHD : 1
 VVVLDDVW : 0

Predição para PHYSALIS

TOP CANDIDATOS: pipelineQ/physalis_transcriptoma.fasta, MODELO: RF

	id	descricao \
12682	J0138557.1.p1	J0138557.1.p1 GENE.J0138557.1~~J0138557.1.p1 ...
13273	J0138990.1.p1	J0138990.1.p1 GENE.J0138990.1~~J0138990.1.p1 ...
9437	J0133072.1.p1	J0133072.1.p1 GENE.J0133072.1~~J0133072.1.p1 ...
9438	J0133073.1.p1	J0133073.1.p1 GENE.J0133073.1~~J0133073.1.p1 ...
13163	J0138904.1.p1	J0138904.1.p1 GENE.J0138904.1~~J0138904.1.p1 ...
12367	J0138307.1.p1	J0138307.1.p1 GENE.J0138307.1~~J0138307.1.p1 ...
12406	J0138325.1.p1	J0138325.1.p1 GENE.J0138325.1~~J0138325.1.p1 ...
857	J0125511.1.p1	J0125511.1.p1 GENE.J0125511.1~~J0125511.1.p1 ...
863	J0125513.1.p1	J0125513.1.p1 GENE.J0125513.1~~J0125513.1.p1 ...
859	J0125512.1.p1	J0125512.1.p1 GENE.J0125512.1~~J0125512.1.p1 ...
2137	J0126343.1.p1	J0126343.1.p1 GENE.J0126343.1~~J0126343.1.p1 ...
2139	J0126345.1.p1	J0126345.1.p1 GENE.J0126345.1~~J0126345.1.p1 ...
2140	J0126346.1.p1	J0126346.1.p1 GENE.J0126346.1~~J0126346.1.p1 ...
2143	J0126349.1.p1	J0126349.1.p1 GENE.J0126349.1~~J0126349.1.p1 ...
2138	J0126344.1.p1	J0126344.1.p1 GENE.J0126344.1~~J0126344.1.p1 ...

```

2141    J0126347.1.p1  J0126347.1.p1  GENE.J0126347.1~~J0126347.1.p1  ...
876     J0125517.1.p1  J0125517.1.p1  GENE.J0125517.1~~J0125517.1.p1  ...
8851    J0132546.1.p1  J0132546.1.p1  GENE.J0132546.1~~J0132546.1.p1  ...
10807   J0134175.1.p1  J0134175.1.p1  GENE.J0134175.1~~J0134175.1.p1  ...
10806   J0134174.1.p1  J0134174.1.p1  GENE.J0134174.1~~J0134174.1.p1  ...

```

	protein_length	probabilidade_resistencia
12682	888	0.964804
13273	885	0.956027
9437	938	0.951206
9438	938	0.951206
13163	884	0.944430
12367	1043	0.940778
12406	868	0.940564
857	833	0.938860
863	833	0.938860
859	833	0.938860
2137	900	0.938606
2139	900	0.938606
2140	873	0.938310
2143	873	0.938310
2138	900	0.937006
2141	900	0.937006
876	833	0.936284
8851	776	0.936274
10807	892	0.936211
10806	892	0.936211

VALIDAÇÃO BIOLÓGICA

```

P_LOOP : 9
KINASE2 : 8
RNBS_A : 1
RNBS_B : 9
RNBS_C : 11
GLPL : 7
RNBS_D : 9
MHD : 0
VVVLDDVW : 0

```

Arquivo salvo:
pipelineQ/predicoes_rf.csv

```

[14]: '''
      Aplicação - Modelo SVM
      '''

```

```

# print("\nPredição para ARABIDOPSIS\n")
# predizer_transcritos(transcriptoma_arabidopsis_fasta,
↳svm_model, "SVM", output="predicoes_svm.csv")

# print("\nPredição para BETACEUM\n")
# predizer_transcritos(transcriptoma_betaceum_fasta,
↳svm_model, 'SVM', output="predicoes_svm.csv")

print("\nPredição para MELONGENA\n")
predizer_transcritos(transcriptoma_melongena_fasta,
↳svm_model, 'SVM', output="predicoes_svm.csv")

print("\nPredição para TUBEROSUM\n")
predizer_transcritos(transcriptoma_tuberosum_fasta,
↳svm_model, "SVM", output="predicoes_svm.csv")

print("\nPredição para PHYSALIS\n")
predizer_transcritos(transcriptoma_physalis_fasta,
↳svm_model, "SVM", output="predicoes_svm.csv")

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")

```

Predição para MELONGENA

TOP CANDIDATOS: pipelineQ/melongena_transcriptoma.fasta, MODELO: SVM

	id	descricao \
590	GBEF01002242.1.p1	GBEF01002242.1.p1 GENE.GBEF01002242.1~~GBEF010...
589	GBEF01002241.1.p1	GBEF01002241.1.p1 GENE.GBEF01002241.1~~GBEF010...
6390	GBEF01032474.1.p1	GBEF01032474.1.p1 GENE.GBEF01032474.1~~GBEF010...
286	GBEF01001123.1.p1	GBEF01001123.1.p1 GENE.GBEF01001123.1~~GBEF010...
263	GBEF01001032.1.p1	GBEF01001032.1.p1 GENE.GBEF01001032.1~~GBEF010...
8171	GBEF01041955.1.p1	GBEF01041955.1.p1 GENE.GBEF01041955.1~~GBEF010...
4127	GBEF01025346.1.p1	GBEF01025346.1.p1 GENE.GBEF01025346.1~~GBEF010...
994	GBEF01014969.1.p1	GBEF01014969.1.p1 GENE.GBEF01014969.1~~GBEF010...
36	GBEF01000129.1.p1	GBEF01000129.1.p1 GENE.GBEF01000129.1~~GBEF010...
967	GBEF01014919.1.p1	GBEF01014919.1.p1 GENE.GBEF01014919.1~~GBEF010...
9150	GBEF01044247.1.p1	GBEF01044247.1.p1 GENE.GBEF01044247.1~~GBEF010...
3816	GBEF01024662.1.p1	GBEF01024662.1.p1 GENE.GBEF01024662.1~~GBEF010...
5435	GBEF01028464.1.p1	GBEF01028464.1.p1 GENE.GBEF01028464.1~~GBEF010...
3969	GBEF01025015.1.p1	GBEF01025015.1.p1 GENE.GBEF01025015.1~~GBEF010...
516	GBEF01002010.1.p1	GBEF01002010.1.p1 GENE.GBEF01002010.1~~GBEF010...
9152	GBEF01044249.1.p1	GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010...
517	GBEF01002011.1.p1	GBEF01002011.1.p1 GENE.GBEF01002011.1~~GBEF010...
518	GBEF01002012.1.p1	GBEF01002012.1.p1 GENE.GBEF01002012.1~~GBEF010...

4989 GBEF01027318.1.p1 GBEF01027318.1.p1 GENE.GBEF01027318.1~~GBEF010...
 2852 GBEF01022665.1.p1 GBEF01022665.1.p1 GENE.GBEF01022665.1~~GBEF010...

	protein_length	probabilidade_resistencia
590	802	1.000000
589	815	1.000000
6390	905	0.999998
286	749	0.999997
263	412	0.999997
8171	959	0.999996
4127	1022	0.999996
994	1020	0.999994
36	718	0.999994
967	978	0.999994
9150	713	0.999993
3816	462	0.999993
5435	409	0.999992
3969	1126	0.999992
516	750	0.999991
9152	579	0.999990
517	746	0.999989
518	727	0.999988
4989	935	0.999987
2852	741	0.999987

VALIDAÇÃO BIOLÓGICA

P_LOOP : 0
 KINASE2 : 5
 RNBS_A : 2
 RNBS_B : 8
 RNBS_C : 8
 GLPL : 2
 RNBS_D : 3
 MHD : 0
 VVVLDDVW : 0

Predição para TUBEROSUM

TOP CANDIDATOS: pipelineQ/tuberosum_transcriptoma.fasta, MODELO: SVM

	id	descricao \
211	MG763883.1.p1	MG763883.1.p1 GENE.MG763883.1~~MG763883.1.p1 ...
46	EF471737.1.p1	EF471737.1.p1 GENE.EF471737.1~~EF471737.1.p1 ...
45	EF137705.1.p1	EF137705.1.p1 GENE.EF137705.1~~EF137705.1.p1 ...
163	KC914392.1.p1	KC914392.1.p1 GENE.KC914392.1~~KC914392.1.p1 ...
277	PQ276083.1.p1	PQ276083.1.p1 GENE.PQ276083.1~~PQ276083.1.p1 ...
162	KC914391.1.p1	KC914391.1.p1 GENE.KC914391.1~~KC914391.1.p1 ...

40	AY704177.1.p1	AY704177.1.p1	GENE.AY704177.1~~AY704177.1.p1	...
43	BD391474.1.p1	BD391474.1.p1	GENE.BD391474.1~~BD391474.1.p1	...
269	PP061819.1.p1	PP061819.1.p1	GENE.PP061819.1~~PP061819.1.p1	...
152	JE864385.1.p1	JE864385.1.p1	GENE.JE864385.1~~JE864385.1.p1	...
78	HI543298.1.p1	HI543298.1.p1	GENE.HI543298.1~~HI543298.1.p1	...
23	AJ306626.1.p1	AJ306626.1.p1	GENE.AJ306626.1~~AJ306626.1.p1	...
262	PP061818.1.p2	PP061818.1.p2	GENE.PP061818.1~~PP061818.1.p2	...
253	PP061808.1.p2	PP061808.1.p2	GENE.PP061808.1~~PP061808.1.p2	...
252	PP061795.1.p2	PP061795.1.p2	GENE.PP061795.1~~PP061795.1.p2	...
24	AJ306627.1.p1	AJ306627.1.p1	GENE.AJ306627.1~~AJ306627.1.p1	...
128	JC300104.1.p1	JC300104.1.p1	GENE.JC300104.1~~JC300104.1.p1	...
137	JC300327.1.p1	JC300327.1.p1	GENE.JC300327.1~~JC300327.1.p1	...
273	PP061822.1.p2	PP061822.1.p2	GENE.PP061822.1~~PP061822.1.p2	...
173	KT036486.1.p1	KT036486.1.p1	GENE.KT036486.1~~KT036486.1.p1	...

	protein_length	probabilidade_resistencia
211	895	0.997342
46	1207	0.994923
45	849	0.994401
163	616	0.991725
277	603	0.990293
162	616	0.987104
40	350	0.986765
43	871	0.984325
269	337	0.983258
152	427	0.977587
78	427	0.977587
23	677	0.971690
262	337	0.968865
253	337	0.968865
252	337	0.968865
24	677	0.965828
128	483	0.962990
137	483	0.962990
273	337	0.960700
173	618	0.960472

VALIDAÇÃO BIOLÓGICA

P_LOOP : 7
KINASE2 : 2
RNBS_A : 5
RNBS_B : 8
RNBS_C : 5
GLPL : 6
RNBS_D : 8
MHD : 1
VVVLDDVW : 0

Predição para PHYSALIS

TOP CANDIDATOS: pipelineQ/physalis_transcriptoma.fasta, MODELO: SVM

	id		descricao \
7040	J0130274.1.p1	J0130274.1.p1	GENE.J0130274.1~~J0130274.1.p1 ...
7039	J0130273.1.p1	J0130273.1.p1	GENE.J0130273.1~~J0130273.1.p1 ...
8851	J0132546.1.p1	J0132546.1.p1	GENE.J0132546.1~~J0132546.1.p1 ...
8854	J0132547.1.p1	J0132547.1.p1	GENE.J0132547.1~~J0132547.1.p1 ...
12868	J0138683.1.p1	J0138683.1.p1	GENE.J0138683.1~~J0138683.1.p1 ...
7042	J0130276.1.p1	J0130276.1.p1	GENE.J0130276.1~~J0130276.1.p1 ...
8855	J0132548.1.p1	J0132548.1.p1	GENE.J0132548.1~~J0132548.1.p1 ...
13597	J0139229.1.p1	J0139229.1.p1	GENE.J0139229.1~~J0139229.1.p1 ...
7041	J0130275.1.p1	J0130275.1.p1	GENE.J0130275.1~~J0130275.1.p1 ...
8857	J0132549.1.p1	J0132549.1.p1	GENE.J0132549.1~~J0132549.1.p1 ...
12367	J0138307.1.p1	J0138307.1.p1	GENE.J0138307.1~~J0138307.1.p1 ...
725	J0125398.1.p1	J0125398.1.p1	GENE.J0125398.1~~J0125398.1.p1 ...
9671	J0133202.1.p1	J0133202.1.p1	GENE.J0133202.1~~J0133202.1.p1 ...
941	J0125565.1.p1	J0125565.1.p1	GENE.J0125565.1~~J0125565.1.p1 ...
724	J0125397.1.p1	J0125397.1.p1	GENE.J0125397.1~~J0125397.1.p1 ...
3431	J0127271.1.p1	J0127271.1.p1	GENE.J0127271.1~~J0127271.1.p1 ...
3428	J0127269.1.p1	J0127269.1.p1	GENE.J0127269.1~~J0127269.1.p1 ...
712	J0125392.1.p1	J0125392.1.p1	GENE.J0125392.1~~J0125392.1.p1 ...
710	J0125391.1.p1	J0125391.1.p1	GENE.J0125391.1~~J0125391.1.p1 ...
9672	J0133203.1.p1	J0133203.1.p1	GENE.J0133203.1~~J0133203.1.p1 ...

	protein_length	probabilidade_resistencia
7040	632	1.000000
7039	539	1.000000
8851	776	1.000000
8854	759	1.000000
12868	1148	0.999999
7042	594	0.999999
8855	639	0.999999
13597	808	0.999999
7041	501	0.999998
8857	622	0.999998
12367	1043	0.999998
725	1171	0.999998
9671	466	0.999998
941	646	0.999998
724	1171	0.999998
3431	887	0.999998
3428	887	0.999998
712	1052	0.999998
710	1052	0.999998
9672	466	0.999997

VALIDAÇÃO BIOLÓGICA

P_LOOP : 5
KINASE2 : 10
RNBS_A : 0
RNBS_B : 1
RNBS_C : 6
GLPL : 6
RNBS_D : 5
MHD : 1
VVVLDDVW : 0

Arquivo salvo:
pipelineQ/predicoes_svm.csv

[]: